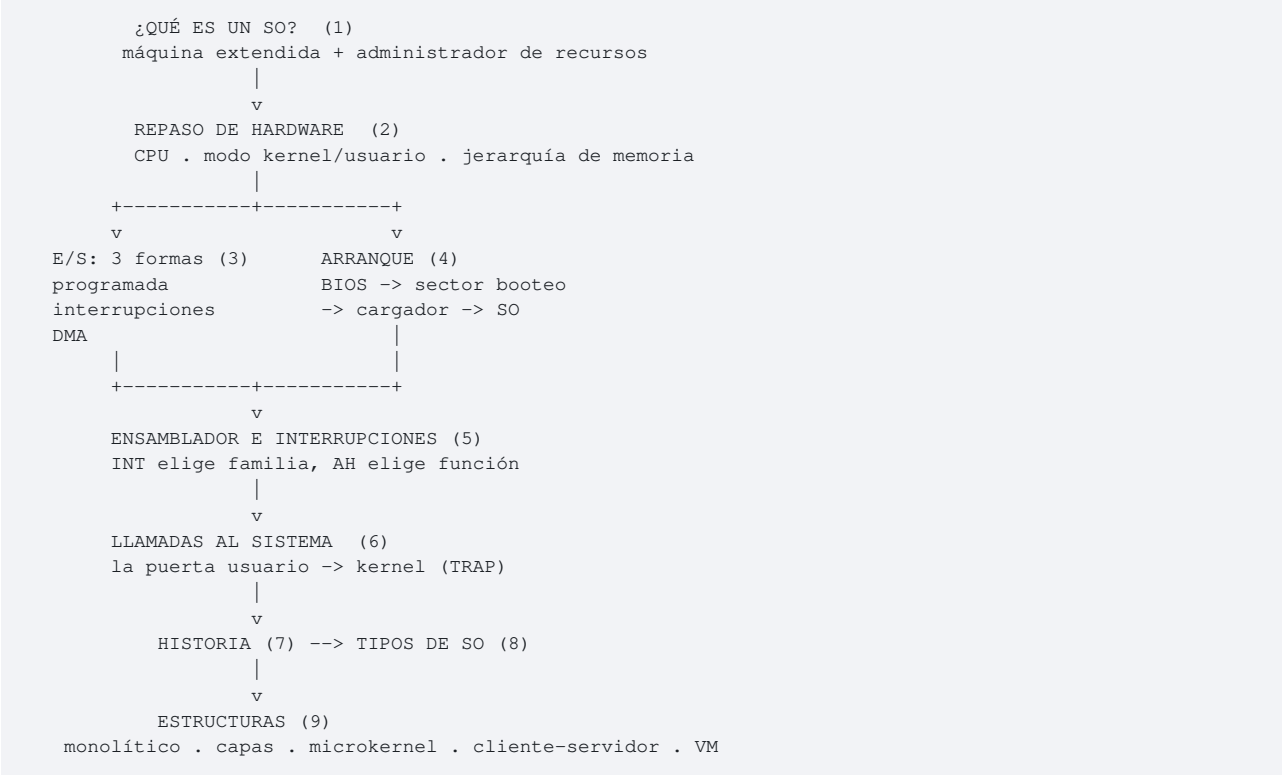
Unidad 1 — Introducción, hardware y estructura de los SO

Sistemas Operativos — UTN FRM — Repaso de la clase completa, 9 temas

La idea que atraviesa toda la unidad: el sistema operativo existe para esconder un hardware incómodo y para repartirlo entre varios que lo quieren al mismo tiempo. Los temas 2 a 5 son *el hardware que hay que esconder*. El tema 6 es *la puerta por donde se pide*. Los temas 7 a 9 son *cómo se fue organizando por dentro*.

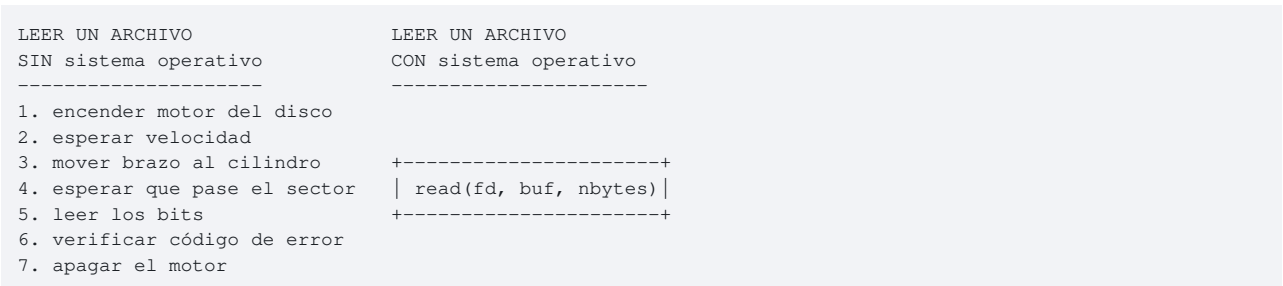


Tema 1 — ¿Qué es un sistema operativo?

Es difícil definirlo más allá de decir que es el software que se ejecuta en modo kernel, y ni siquiera eso es siempre cierto. El problema es que cumple **dos funciones básicas que no están relacionadas entre sí**.

Función 1 — Máquina extendida

La arquitectura de la mayoría de las computadoras a nivel de lenguaje máquina es **primitiva y compleja de programar**. La **abstracción** es la clave para lidiar con esa complejidad, y la abstracción que todo el mundo entiende es el **archivo**.



"El trabajo del sistema operativo es crear buenas abstracciones para después implementar y administrar los

objetos abstractos entonces creados."

Función 2 — Administrador de recursos

Proporciona una **asignación ordenada y controlada** de los procesadores, memorias y dispositivos de E/S entre los programas que compiten por ellos. Lo hace **multiplexando** de dos formas:

	Multiplexaje en el TIEMPO	Multiplexaje en el ESPACIO
Idea	Toman turnos	Cada uno recibe una parte
Recursos	CPU, impresora	Memoria principal, disco
Problema que genera	¿Quién va primero? → planificación (U2)	¿Y si uno pisa al otro? → protección (U3)

Caso del disco: se multiplexa en el **espacio** (cada usuario sus bloques), pero adentro hay **dos recursos distintos**: la superficie magnética se reparte en espacio, y el **brazo**, que es uno solo, se multiplexa en el **tiempo**. De ahí salen los algoritmos de planificación de disco de la Unidad 5.

Ejemplo resuelto: por qué la multiprogramación mejora el uso de la CPU

Un proceso pasa el 80 % de su tiempo esperando E/S. ¿Cuánto se aprovecha la CPU según cuántos procesos haya?

Modelo: si cada proceso espera E/S una fracción `p` del tiempo y hay `n` procesos independientes, la probabilidad de que los n estén esperando a la vez es `p^n`. Entonces el aprovechamiento es **1 – p^n**.

Procesos en memoria	Aprovechamiento de CPU
1	20,00 %
2	36,00 %
3	48,80 %
4	59,04 %
5	67,23 %
10	89,26 %

El rendimiento marginal decrece: cada proceso adicional aporta menos que el anterior y además consume memoria.

Referencia: Tanenbaum, cap. 1.1, págs. impresas 3–7 → PDF 33–37. Y cap. 2.1.7, impresas 93–95.

Tema 2 — Repaso del hardware

La CPU y sus registros

Ciclo básico de toda CPU: **obtener** la instrucción de memoria, **decodificarla** para determinar su tipo y operandos, **ejecutarla**, y volver a empezar.

Registro	Qué contiene	Por qué le importa al SO
Contador de programa (PC)	Dirección de la siguiente instrucción a obtener	Para suspender y reanudar un proceso
Apuntador de pila (SP)	Tope de la pila: valores de los procedimientos llamados y no retornados	Cada proceso tiene su propia pila

PSW (palabra de estado)	Bits de código de condición **y el bit de modo**	Acá vive la protección

Modo kernel y modo usuario — el cimiento de todo

```
+-------------------------------------------------+
|  MODO USUARIO                                   |
|  programas comunes                              |
|  . solo un SUBCONJUNTO de instrucciones         |
|  . solo un SUBCONJUNTO del hardware             |
|  . NO puede: apagar interrupciones, tocar E/S,  |
|    acceder a memoria ajena                      |
+--------------------+----------------------------+
                     |
                   TRAP        <- única puerta legal
                     |
                     v
+-------------------------------------------------+
|  MODO KERNEL                                    |
|  el sistema operativo                           |
|  . CUALQUIER instrucción del repertorio         |
|  . TODO el hardware                             |
+-------------------------------------------------+
```

Error típico que hay que evitar: si un programa de usuario ejecuta una instrucción privilegiada (apagar interrupciones, tocar registros de E/S, cambiar la MMU), **NO** es que el sistema operativo se cuelgue y después reaccione. El **hardware** verifica el bit de modo del PSW y **rechaza la instrucción antes de ejecutarla**, generando un trap. Recién ahí el SO toma el control y normalmente mata al proceso.
La protección la impone el hardware; el sistema operativo la configura.

La jerarquía de memoria

```
       ^ más rápido, más caro, menos capacidad
       |
  Registros      ~igual que la CPU      < 1 KB      volátil
  Caché L1/L2    1 a 2 ciclos           KB a MB     volátil
  RAM            decenas de ns          GB          volátil
  Disco          milisegundos           TB          NO volátil
  Cinta          segundos               enorme      NO volátil
       |
       v más lento, más barato, más capacidad
```

Cómo funciona la caché: la memoria principal se divide en **líneas de caché**. Las más usadas se mantienen en una caché rápida dentro o muy cerca de la CPU. Si la línea pedida está, hay **acierto de caché** y no se envía petición por el bus a la memoria principal. Si no, hay **fallo** y se paga el acceso completo. La L1 está dentro de la CPU y se accede sin retraso; la L2 tiene varios MB y cuesta uno o dos ciclos de reloj.

Ejemplo resuelto: tiempo medio de acceso con caché

Caché L1 = 1 ns, RAM = 10 ns, tasa de aciertos = 95 %.

```
Paso 1   95% de las veces:  está en caché       ->  1 ns
Paso 2    5% de las veces:  fallo               ->  1 ns (mirar) + 10 ns (RAM) = 11 ns
Paso 3   promedio ponderado:
         Tm = 0,95 x 1  +  0,05 x 11  =  0,95 + 0,55  =  1,50 ns
Paso 4   sin caché sería 10 ns  ->  10 / 1,50 = 6,67 veces más rápido
```

Variante con mala localidad (50 % de aciertos): Tm = 0,5 × 1 + 0,5 × 11 = **6,00 ns**, mejora de solo **1,67×**. La caché **vale lo que vale la localidad del programa**: el hardware es el mismo, lo que cambió es el software.

El resto

- **Memoria no volátil:** ROM (de fábrica, no se modifica), EEPROM y flash (sí se reescriben), CMOS (volátil,

con batería: fecha, hora y configuración del BIOS).

- **Disco:** platos que giran a 5400, 7200 o 10800 rpm. **Pista** = región anular que lee una cabeza. **Cilindro** = todas las pistas para una misma posición del brazo.
- **MMU** (Memory Management Unit): la parte de la CPU que traduce la dirección que genera el programa en la dirección física real. Es el hardware de la memoria virtual (U3).
- **Dispositivos de E/S:** siempre son dos partes, el dispositivo + su **controlador** (chip). El software que le habla al controlador es el **driver**, y corre en modo kernel.
- **Buses:** ISA (viejo) → PCI; USB para dispositivos lentos; SCSI para dispositivos veloces.
- **Plug and play:** permite que el sistema recolecte automáticamente la información de los dispositivos y asigne de forma centralizada los niveles de interrupción y las direcciones de E/S.

Referencia: Tanenbaum, cap. 1.3, págs. impresas 19–33 → PDF 49–63.

Tema 3 — Las tres formas de hacer E/S

Lo más importante del tema: los tres métodos transfieren los mismos datos y **tardan exactamente lo mismo**, porque la duración la impone el dispositivo. Lo que cambia radicalmente es **cuánta CPU se gasta**. La trampa de examen es preguntar "¿cuál es más rápido?": **ninguno**.

1. E/S programada (espera ocupada / polling)

El driver inicia la operación y entra en un **ciclo cerrado sondeando continuamente** al dispositivo para ver si terminó. Mantiene la CPU ocupada durante toda la transferencia.

```
CPU:  [sondea][sondea][sondea][sondea][sondea]  <- 100% quemada
DISP: [============ transfiriendo ===========]
```

2. E/S controlada por interrupciones

El driver inicia el dispositivo, le pide que **genere una interrupción cuando termine**, y retorna. El SO bloquea al proceso llamador si hace falta y busca otro trabajo. Paga una interrupción por unidad de datos.

```
CPU:  [otros procesos][i][otros procesos][i][otros...][i]
DISP: [===byte===][===byte===][===byte===][===byte===]
      i = interrupción
```

3. E/S con DMA (Direct Memory Access)

Un chip especial controla el flujo de bits entre la memoria y el controlador **sin intervención constante de la CPU**. La CPU lo configura (cuántos bytes, qué direcciones, qué operación) y lo deja trabajar. Al terminar, genera **una única interrupción**.

```
CPU:  [c][======= otros procesos ===========][i]
       ^                                     ^
       configura el DMA                1 sola interrupción
DISP: [============ el DMA transfiere =========]
```

Canales de datos

En las mainframes la idea se lleva al extremo: procesadores de E/S especializados, con su propio repertorio de instrucciones, que ejecutan programas de canal completos y manejan muchos dispositivos de forma autónoma.

Comparación

	Programada	Interrupciones	DMA
¿CPU libre durante la espera?	No	Sí	Sí

Interrupciones	0	una por byte o bloque	1
Hardware extra	ninguno	controlador de interrupciones	chip DMA
Dónde se usa	dispositivos simples, arranque	teclado, mouse	disco, red, video

Ejemplo resuelto: el que se toma

1000 bytes, el dispositivo entrega un byte cada 100 µs, atender una interrupción cuesta 5 µs.

```
Paso 1  Duración total = 1000 x 100 us = 100.000 us = 100 ms
        IGUAL EN LOS TRES. La impone el dispositivo.

Paso 2  PROGRAMADA   -> CPU = 100 ms           ->  100 %
Paso 3  INTERRUPC.   -> 1000 x 5 us = 5 ms     ->  5 %
Paso 4  DMA          -> 1 x 5 us = 0,005 ms    ->  0,005 %
```

Por qué el teclado usa interrupciones y el disco DMA:
Teclado escribiendo rápido, unas 10 teclas/s → 10 × 5 µs = 50 µs de CPU por segundo = **0,005 %**. Perfecto.
Disco a 100 MB/s con una interrupción por byte → 100.000.000 × 5 µs = **500 segundos de CPU por cada segundo**. Imposible. Por eso DMA.
Ojo: el disco **sí usa interrupciones**, pero una por transferencia completa. El DMA no reemplaza a las interrupciones: **reduce su cantidad a una sola**.

Referencia: Tanenbaum, cap. 1.3.5, impresas 27–30 → PDF 57–60; y cap. 5.2, impresas 343–348 → PDF 373–378.

Tema 4 — El arranque y el sector de booteo

El problema del huevo y la gallina: el sistema operativo está en el disco, pero para leer el disco hace falta el sistema operativo. La solución es una **cadena de cargadores**, cada uno más grande y más capaz que el anterior, y cada uno existe solo para poder cargar al siguiente.

Qué es una partición

Un disco es una **tira continua de sectores numerados**. Particionarlo es como levantar paredes dentro de un galpón: las paredes **no existen físicamente**, son **información anotada** en el primer sector.

```
        GALPÓN (= el disco físico)
+---------------+-----------------+--------------+------------+
|   Depósito 1  |   Depósito 2    |  Depósito 3  | Depósito 4 |
|   Windows     |     Linux       |   tus fotos  |   vacío    |
|   (NTFS)      |     (ext4)      |   (FAT32)    |            |
+---------------+-----------------+--------------+------------+
   cada uno con su organización interna, sin verse entre sí
```

La cadena de arranque

```
chip en la placa          sector 0 del disco      primer sector de
(no depende del disco)    446 bytes               la partición activa      archivo en la partición
      |                        |                        |                        |
      v                        v                        v                        v
   +------+   lee 512b    +--------+   ¿cuál      +-------------+  "buscá el   +----------+
   | BIOS | ------------> |  MBR   | --es activa?->| CARGADOR    | --kernel"--> |    SO    |
   +------+               |        |               | SECUNDARIO |              +----------+
   no sabe                +--------+               +-------------+              ya sabe todo
   leer archivos          sabe leer                sabe leer
                          el plano                 archivos

   cada eslabón es más grande y más capaz que el anterior,
   y cada uno existe solo para poder cargar al siguiente
```

Los siete pasos

1. Se enciende. La CPU ejecuta el **BIOS**, que está en ROM/flash en la motherboard (por eso no depende del disco).
2. El BIOS cuenta la RAM, verifica el teclado, explora los buses ISA y PCI y registra los dispositivos heredados y los plug and play.
3. Determina el dispositivo de arranque consultando la lista guardada en la **memoria CMOS**.
4. Lee el **primer sector**, lo carga en memoria y lo ejecuta. Ese es el **sector de booteo**.
5. El sector de booteo examina su **tabla de particiones**, busca cuál está **activa**, y lee de ahí el **cargador secundario**.
6. El cargador secundario lee el sistema operativo y lo inicia.
7. El SO consulta al BIOS la configuración, carga los drivers en el kernel, inicializa sus tablas, crea los procesos de segundo plano y arranca el login o la GUI.

Ejemplo resuelto: adentro de los 512 bytes

```
SECTOR DE BOOTEO MAESTRO (MBR) — un sector = 512 bytes

byte 0                                  445  446      509 510  511
+-----------------------------------------++----------++--------+
|      CÓDIGO DE ARRANQUE                 ||  TABLA   || 55 AA  |
|      446 bytes                          ||    DE    || firma  |
|      (en ensamblador, por el tamaño)    ||PARTICION.|| 2 bytes|
+-----------------------------------------++----------++--------+
                                            4 entradas
                                            x 16 bytes
                                            = 64 bytes

Paso 1   Firma 55AA al final                      =   2 bytes
Paso 2   Tabla: 4 particiones primarias x 16      =  64 bytes
Paso 3   Lo que sobra para el código: 512-64-2    = 446 bytes
Paso 4   Verificación: 446 + 64 + 2               = 512  OK
```

La confusión más común: los 446 son poco y los 64 alcanzan de sobra. ¿Por qué?
Porque **no se comparan**: los 446 bytes son **un programa** (instrucciones que se ejecutan) y los 64 bytes son **datos** (cuatro renglones de números).

La tabla de particiones no contiene las particiones: las describe. El plano del galpón es una hoja A4, aunque el galpón mida 2000 m². Un renglón de 16 bytes describe una partición de **cualquier** tamaño:

```
UN RENGLÓN = 16 bytes:
   1 byte  -> ¿es la activa? (sí/no)
   1 byte  -> tipo (NTFS, ext4, FAT32...)
   4 bytes -> sector donde EMPIEZA
   4 bytes -> cuántos sectores MIDE
   6 bytes -> otros datos (geometría)
```

Partición de 50 MB → 16 bytes. De 2 TB → 16 bytes. De 500 TB → 16 bytes. **Siempre 16.**
Y una tabla completa de 4 particiones = 4 × 16 = **64 bytes**, sin importar los tamaños.

Dos datos que explican cosas:
• Sin la firma `55AA`, el BIOS considera que el disco **no es arrancable**. Una corrupción de 2 bytes deja la máquina sin bootear aunque todos los archivos estén intactos.
• El **límite de 2 TB por partición** del MBR viene de que el campo "cuántos sectores mide" tiene solo 4 bytes: $2^{32} - 1$ sectores × 512 bytes ≈ 2 TB. **El límite no viene del disco, viene del tamaño del casillero.** Para superarlo se inventó GPT, con campos de 8 bytes.

¿Cómo el BIOS lee el sector 0 si no sabe leer archivos? Porque son dos niveles distintos. "Dame el sector

0" es **una orden al hardware**, que la controladora del disco entiende sola. "Dame el archivo /boot/vmlinuz" es una **abstracción construida por software** sobre miles de sectores, que exige conocer el formato exacto de ese sistema de archivos. El BIOS sabe abrir el libro en la página 237; no sabe buscar un capítulo por el índice. Por eso puede arrancar cualquier disco sin saber qué sistema de archivos tiene: nunca lee un archivo.

Referencia: Tanenbaum, cap. 1.3.7, pág. impresa 33 → PDF 63. El reparto interno de los 512 bytes del MBR es un complemento elaborado: el libro describe el proceso pero no la estructura del sector.

Tema 5 — Ensamblador e interrupciones del 8086

El problema de fondo: en ensamblador **no existe una instrucción para mostrar texto en pantalla**. Las instrucciones del procesador son mover, sumar, restar, comparar y saltar. Nada más.
La solución: alguien ya escribió esas rutinas (el BIOS y el DOS) y están en memoria desde que arranca la máquina. Vos solo las llamás.

La regla de oro

`INT xxh` elige la FAMILIA. `AH` elige la FUNCIÓN dentro de esa familia. Los demás registros llevan los datos.

El apunte de la cátedra avisa de la trampa: *"No todo va en AH: AH selecciona la función; los datos pueden ir en AL, DL, DX, BX, CX, DS:DX, etc."*

```
      VOS                                      DOS
      ---                                      ---
  pongo en DX  ->  +---------------+
  la dirección     | DX = dónde    |
  del texto        | está el texto |
                   +---------------+
  pongo en AH  ->  | AH = 09h      |
  el nº de         | (mostrar      |
  trámite          |  cadena)      |
                   +-------+-------+
                           |
  ejecuto INT 21h  --------+--------->  "me llamaron"
                                        mira AH -> 09h -> "quiere mostrar
                                        una cadena"
                                        mira DX -> va a esa dirección,
                                        lee el texto y lo pinta
```

Los registros

```
    +------------------+
AX  |   AH   |   AL    |   dos mitades de 8 bits, direccionables por separado
BX  |   BH   |   BL    |   AH = la de arriba (high), AL = la de abajo (low)
CX  |   CH   |   CL    |
DX  |   DH   |   DL    |
    +------------------+
       16 bits total

DS, ES  -> registros de SEGMENTO
SI, DI, BP, SP -> índice y punteros
```

Qué significa `DS:DX`: es una sola dirección partida en dos, como una dirección postal. `DS` es el **segmento** (en qué bloque de memoria están los datos) y `DX` el **desplazamiento** (a cuántos bytes del inicio de ese bloque). En un programa `.COM` con `ORG 100h`, `DS` ya viene cargado; en un `.EXE` hay que cargarlo, y como no se admite un valor inmediato en un registro de segmento, se pasa por `AX`: `MOV AX, @data` y después `MOV DS, AX`.

INT 21h — funciones de DOS (la más importante)

AH	Qué hace	Dónde van los datos

01h	Lee un carácter del teclado CON eco	retorna en AL
02h	**Muestra UN carácter**	**el carácter va en DL**
07h	Lee sin eco	retorna en AL
08h	Lee sin eco + controla Ctrl-Break	retorna en AL
09h	**Muestra una CADENA**	**DS:DX, termina en $**
0Ah	Lee una cadena a un buffer	DS:DX (byte 0 capacidad, byte 1 leídos, byte 2+ caracteres)
0Bh	¿Hay entrada disponible?	AL = FFh sí, 00h no
2Ah / 2Ch	Fecha / Hora	CX año, DH mes, DL día / CH:CL:DH
3Ch–42h	Archivos: crear, abrir, cerrar, leer, escribir, borrar, seek	BX handle, CX cantidad, DS:DX buffer
4Ch	**Termina el proceso**	AL = código de retorno (o `MOV AX, 4C00h`)

INT 10h — video

AH	Qué hace	Datos	¿Avanza el cursor?
02h	Posicionar cursor	DH fila, DL columna, BH página	—
09h	Carácter con atributo (color)	AL char, BL atributo, CX cantidad	**NO**
0Ah	Carácter sin atributo nuevo	AL char, CX cantidad	**NO**
0Eh	Carácter y avanza	AL char	**SÍ**
13h	Cadena completa	ES:BP dirección, CX **longitud**, DH/DL posición, BL atributo	—

La cartilla marca que 09h, 0Ah, 0Eh y 13h son **las más fáciles de confundir**.

INT 16h — teclado (BIOS)

- **00h** — lee tecla y **espera** hasta que haya una. Retorna AL = ASCII, AH = scan code.
- **01h** — **consulta** si hay tecla, no bloquea. ZF=0 hay, ZF=1 no hay.
- **02h** — estado de Shift, Ctrl, Alt, Caps Lock, Num Lock, en AL.

Fijate el paralelo con el tema 3: 00h **bloquea**, 01h **sondea** (polling).

INT 17h — impresora

00h imprimir carácter (AL, con DX=0 para LPT1 y DX=1 para LPT2), 01h inicializar, 02h obtener estado (retorna en AH).

Ejemplo resuelto: el ejercicio del final práctica

```
ORG 100h
JMP MAIN
MSG  DB  'ESTA ES LA CADENA A MOSTRAR$'
MAIN:
     MOV DX, MSG      ; dónde está el texto
     MOV AH, 09h      ; trámite: mostrar cadena
     INT 21
     RET
```

Paso 1. Hay que mostrar una CADENA, no un carácter → familia INT 21h, función AH = 09h.

Paso 2. La 09h pide la dirección en DS:DX y la cadena terminada en `$`.
Paso 3. `MOV DX, MSG` carga el desplazamiento.
Paso 4. `MOV AH, 09h` selecciona la función.
Paso 5. `INT 21` ejecuta: DOS recorre desde DS:DX hasta encontrar el `$`.
Paso 6. `RET` vuelve a DOS (en un .EXE sería `MOV AX, 4C00h` + `INT 21h`).

Los 4 errores típicos que marca la cátedra:

```
X usar DL en lugar de DX        -> DL es para UN carácter (02h), no para cadenas
X olvidar el $ final            -> DOS sigue mostrando basura de memoria
                                   hasta encontrar un $ por casualidad
X no cargar DS                  -> la dirección apunta a cualquier lado
X usar AH=02h para una cadena   -> muestra un solo carácter
```

Sobre el segundo: sin el `$`, DOS no sabe que el texto terminó y sigue leyendo bytes de memoria —restos de otros programas— interpretándolos como caracteres. Podrían ser 5 o 500 caracteres de basura.

Dos aclaraciones útiles:
• **El código se ejecuta de arriba hacia abajo**, siempre. Se explica al revés (empezando por el INT) porque esa es la acción, y las líneas de arriba son la preparación. Como una receta: poner la olla, prender el fuego, tirar los fideos.
• `INT` hace algo muy parecido a un `CALL`: apila la dirección de retorno y el PSW, salta a la rutina, y al final la rutina hace `IRET` que desapila todo. Es **un CALL con control de seguridad**.

Referencia: Apunte *Interrupciones (1).pdf* y *Tp5 - Tabla Interrupciones.pdf* (Cartilla de Interrupciones UTN-FRM). Este tema no está en el Tanenbaum: es material propio de la cátedra.

Tema 6 — Llamadas al sistema

Definición: las llamadas al sistema son **la interfaz que permite a los programas de usuario solicitar servicios al sistema operativo**. Son la frontera exacta entre el modo usuario y el modo kernel.

Lo que hiciste con `INT 21h` **ya era una llamada al sistema** (de DOS). El mecanismo es el mismo.

La instrucción TRAP

	Llamada a procedimiento (CALL)	TRAP
¿Guarda dirección de retorno?	Sí, en la pila	Sí
¿A dónde salta?	A **cualquier** dirección	A una dirección **FIJA**
¿Cambia el modo?	**No**, seguís en usuario	**Sí**, pasás a kernel

Esas **dos diferencias** son toda la seguridad. Si TRAP pudiera saltar a cualquier lado, un programa entraría al kernel por donde se le antoje. Como solo puede saltar a un punto fijo, el kernel controla la entrada.
Una llamada a procedimiento te mueve *dentro de tu casa*; un TRAP te lleva *a la oficina del gobierno*, y solo por la puerta principal.

Llamadas a procedimiento: CALL, RET y la pila

Un procedimiento no sabe quién lo llamó. Si lo llamaste desde la línea 10 tiene que volver a la 11; si lo llamaste desde la 200, a la 201. Se resuelve con la **pila**:

```
CALL hace DOS cosas:
  1. guarda en la PILA la dirección de la línea siguiente  <- "dirección de retorno"
  2. salta al procedimiento
```

```
RET hace UNA cosa:
  1. saca de la PILA esa dirección y salta ahí

    tu programa                       PILA
    línea 10: CALL MOSTRAR   --+   +----------+
    línea 11: sigue acá  <---+ |   |          |
                             | +-->|    11    |  <- CALL guardó esto
                             |     +----------+
    línea 50: MOSTRAR:       |
    línea 52:   RET  --------+
```

Si un procedimiento llama a otro que llama a otro, las direcciones se **apilan** y se desapilan en orden inverso: lo último que entra es lo primero que sale.

Los once pasos de read()

1. 1 a 3 — El programa mete los parámetros en la pila, **en orden inverso** (nbytes, &bufer, fd). El primero y el tercero por valor; el segundo **por referencia** (se pasa la dirección del buffer, no su contenido).
2. 4 — Llamada al procedimiento de biblioteca `read`, una instrucción de llamada común.
3. 5 — El procedimiento de biblioteca (escrito en ensamblador) coloca el **número de la llamada al sistema** en el registro donde el SO lo espera.
4. 6 — Ejecuta **TRAP**: cambia a modo kernel y empieza en una dirección fija del núcleo.
5. 7 — El kernel usa ese número como **índice en una tabla de apuntadores** a manejadores de llamadas.
6. 8 — Se ejecuta el manejador, que hace la lectura real.
7. 9 — El control vuelve al procedimiento de biblioteca, a la instrucción siguiente al TRAP.
8. 10 — El procedimiento de biblioteca retorna al programa de usuario.
9. 11 — El programa limpia la pila: incrementa el apuntador para eliminar los tres parámetros.

Ojo con el paso 9: la llamada puede **bloquear** al proceso. Si se lee del teclado y no se escribió nada, el proceso se bloquea, el SO ejecuta otro, y recién cuando llega la entrada se completan los pasos 9 a 11.

Valor de retorno: devuelve la cantidad de bytes leídos, que puede ser menor que nbytes si se alcanzó el fin de archivo. Si falla, devuelve **–1** y el número de error queda en la variable global **errno**.

Las llamadas POSIX principales

Procesos	Archivos	Directorios y varias
fork — crea un hijo idéntico al padre **execve** — reemplaza la imagen del proceso **exit** — termina y devuelve el estado **waitpid** — espera a que un hijo termine **kill** — envía una señal	**open** — abre un archivo **read** — lee a un buffer **write** — escribe desde un buffer **close** — cierra **lseek** — mueve el puntero **stat** — información del archivo	**mkdir** / **rmdir** **link** / **unlink** **mount** / **umount** **chdir** — cambia el directorio de trabajo **chmod** — cambia los bits de protección **time** — tiempo desde el 1/1/1970

POSIX frente a Win32

En POSIX la correspondencia entre procedimientos de biblioteca y llamadas al sistema es casi **uno a uno**. La API **Win32** de Windows, en cambio, tiene miles de funciones **deliberadamente desacopladas** de las verdaderas llamadas al sistema: son procedimientos de biblioteca que pueden invocar o no al kernel. Esa indirección le permite a Microsoft cambiar el kernel sin romper los programas existentes.

strace — aparece en los exámenes

Sintaxis exacta que piden: `strace -o salida.txt ls -la`

La opción `-o` manda la salida al archivo. Distractores habituales: `>`, `|`, `-s`.

Lo que aparece en ese archivo:

openat()	abre archivos o directorios — en este caso "." , el directorio actual
read()	lee contenido desde un descriptor de archivo
getdents64()	lee las entradas de un directorio: la lista de archivos y carpetas
newfstatat() / statx()	obtiene información de archivos: permisos, tamaño, propietario, fechas
write()	escribe la salida de ls en la terminal
close()	cierra un descriptor y libera recursos del kernel

Ejemplo resuelto: por qué conviene un buffer grande

```
Leer un archivo de 8 KB:

Paso 1   8 KB = 8 x 1024 = 8192 bytes
Paso 2   Con buffer de 512 bytes:   8192 / 512  = 16 llamadas a read()
Paso 3   Con buffer de 4096 bytes:  8192 / 4096 =  2 llamadas a read()
Paso 4   Se leen los MISMOS datos, pero se cruza la frontera
         usuario->kernel 16 veces contra 2.
```

Como cada cruce implica los once pasos, el segundo caso hace **ocho veces menos trabajo de kernel**. Los buffers no ahorran lectura de disco: **ahorran llamadas al sistema**.

Referencia: Tanenbaum, cap. 1.6, págs. impresas 49–62 → PDF 79–92. Y apunte *Comandos_Linux-SO (1).pdf*.

Tema 7 — Historia por generaciones

Las fechas no importan. Lo que se toma es por qué apareció cada invento. Toda la historia es una cadena de **agujeros de tiempo muerto** que se van tapando, porque las computadoras eran carísimas y cada minuto parada era plata tirada.

Agujero 1 — la máquina espera al humano (1945–1955, tubos al vacío)

```
+-------------------------------------------------+
| vos cableando | CORRE | vos cableando | CORRE |  |
|   45 minutos  | 3 min |   50 minutos  | 2 min |  |
+-------------------------------------------------+
   ^                       ^
   la máquina está PARADA mirándote trabajar
```

- Se programaba conectando cables en **tableros de conexiones**. No existían los lenguajes de programación, **ni siquiera el ensamblador**.
- **No había sistemas operativos.**
- El programador se anotaba en una hoja de firmas, entraba al cuarto y esperaba horas rezando que no se quemara ninguno de los 20.000 bulbos.
- A principios de los 50 aparecen las **tarjetas perforadas**: se escribe en tarjetas en vez de cablear.

Antecedente: **Charles Babbage** (1792–1871) diseñó la primera computadora digital verdadera, la máquina analítica, que nunca funcionó por limitaciones mecánicas. Contrató a **Ada Lovelace** como primera programadora.

Agujero 2 — la máquina espera al operador (1955–1965, transistores)

Se saca al programador de la sala y entran los **operadores profesionales**. Pero el operador **camina**: busca tarjetas, carga, saca papel de la impresora, lo lleva al otro cuarto.

Solución: el PROCESAMIENTO POR LOTES. Juntar un montón (un *lote*) de trabajos, pasarlos a una cinta, y

que la máquina los ejecute **uno atrás del otro sin que nadie toque nada**.

```
(a) tarjetas  ->  (b) la 1401 las pasa a cinta  ->  (c) llevan la cinta
                      (máquina barata)                 a mano
(d) la 7094 calcula  ->  (e) llevan la cinta de vuelta  ->  (f) la 1401 imprime
    (máquina cara)
```

Aparecen las **tarjetas de control**: `$JOB` (tiempo máximo, cuenta, programador), `$FORTRAN`, `$LOAD`, `$RUN`, `$END`. **Son las precursoras de los shells e intérpretes de línea de comandos modernos.**

Sistemas operativos típicos: FMS (Fortran Monitor System) e IBSYS.

Agujero 3 — la CPU espera al disco (1965–1980, circuitos integrados)

Ya no hay tiempo muerto **entre** trabajos, pero sí **adentro** de cada uno. En procesamiento de datos comerciales la CPU espera E/S el **80 a 90 %** del tiempo.

```
UN SOLO TRABAJO EN MEMORIA:
CPU:   #.........#.........#.........
       ^ calcula  ^ esperando el disco, sin hacer NADA
```

Solución: la MULTIPROGRAMACIÓN. Partir la memoria y meter varios trabajos a la vez. Cuando uno espera el disco, la CPU se pasa a otro.

```
      MEMORIA
+--------------+
|  Trabajo 3   |
+--------------+        CPU:  ##########################
|  Trabajo 2   |              1  2  3  1  2  3  1  2
+--------------+              siempre hay alguien listo
|  Trabajo 1   |
+--------------+
|      SO      |
+--------------+
```

LOTES vs MULTIPROGRAMACIÓN — la confusión clásica:

PROCESAMIENTO POR LOTES	MULTIPROGRAMACIÓN
UNO en memoria por vez	VARIOS en memoria a la vez
trab1 -> trab2 -> trab3 en fila india	todos juntos
Tapa el agujero ENTRE trabajos	Tapa el agujero DENTRO de un trabajo
(el operador caminando)	(la CPU esperando el disco)

No son lo mismo y no se reemplazan: se suman. Un sistema de tercera generación hace lotes y multiprogramación al mismo tiempo.

Por qué la multiprogramación no era posible antes: con varios trabajos en memoria a la vez, uno puede **pisar la memoria de otro**. Hace falta **hardware de protección**, y recién lo trajo la tercera generación (la IBM 360). Es lo mismo del tema 2: compartir un recurso siempre exige protección.

Otros hitos de la tercera generación:

- **IBM System/360**: la primera **familia de computadoras compatibles** en software. Su SO, el OS/360, era tan enorme que corregir unos errores introducía otros y la cantidad total se mantenía constante.
- **Spooling** (Simultaneous Peripheral Operation On Line): leer los trabajos al disco apenas llegan, para cargar otro apenas una partición queda libre.
- **Tiempo compartido**: cada usuario con su terminal. Si hay 20 conectados y 17 están pensando, la CPU atiende por turnos a los 3 que piden. Funciona porque los comandos interactivos son cortos. Primer sistema: **CTSS** (MIT).
- **MULTICS**: la "utilería para computadora", poder de cómputo como el servicio eléctrico. Éxito parcial

comercialmente, pero **influencia enorme**. Primer sistema con memoria virtual y ligado dinámico.

- **UNIX**: Ken Thompson, que venía de MULTICS, escribió una versión simple en una PDP-7. Se fragmentó en System V y BSD → el IEEE creó el estándar **POSIX** para unificar. Después **MINIX** (Tanenbaum, educativo) y **Linux** (Torvalds, inspirado en MINIX).

Agujero 4 — el usuario espera el resultado

La máquina está al 100 %, pero entregás las tarjetas y el resultado llega en seis horas. Una coma de más te cuesta el día. **Solución: el tiempo compartido**, ya mencionado.

Agujero 5 — ya no hay agujero (1980 a hoy, computadoras personales)

Con los circuitos **LSI** la computadora se vuelve **barata**. Por primera vez el objetivo cambia: ya no es aprovechar la máquina, es **que sea cómoda para la persona**.

- **CP/M** (1974): Intel le encarga un SO a Gary Kildall para el 8080 y le cede los derechos porque no cree en las microcomputadoras con disco. Domina el mercado unos cinco años.
- **MS-DOS** (1980): IBM busca un SO para la PC, Kildall no los atiende, vuelven a Bill Gates. Gates compra DOS a Seattle Computer Products por unos 75.000 dólares, lo rebautiza y se lo ofrece a IBM. **La jugada clave:** vendérselo a las fabricantes para que venga incluido con el hardware, frente al intento de Kildall de venderlo de a un usuario.
- **La interfaz gráfica**: la inventó **Doug Engelbart** (ventanas, iconos, menús, ratón), la adoptó **Xerox PARC**, y Steve Jobs entendió su valor al visitarlos. Lisa fracasó por costosa; la **Macintosh** fue un éxito por económica y *amigable*.
- **Windows**: de 1985 a 1995 fue solo un entorno gráfico encima de MS-DOS. Windows 95 fue independiente. Windows NT fue una reescritura de cero en 32 bits, y es la línea de la que descienden 2000, XP y Vista.

El patrón que se toma: *"la ontogenia recapitula la filogenia"*. Ideas que se inventaron, se descartaron cuando el contexto cambió, **y volvieron una década después en otro contexto**: la programación en ensamblador, la monoprogramación, los directorios de un solo nivel. Desaparecieron de las máquinas grandes y reaparecieron en los sistemas embebidos y las tarjetas inteligentes.

La historia completa en un cuadro

Agujero	¿Quién esperaba?	Solución
1	La máquina esperaba al programador	Sacarlo de la sala: **operadores**
2	La máquina esperaba al operador	**Procesamiento por lotes**
3	La **CPU** esperaba al **disco**	**Multiprogramación** + spooling
4	El **usuario** esperaba el resultado	**Tiempo compartido**
5	Ya no importa: la máquina es barata	**Computadora personal** y GUI

Referencia: Tanenbaum, cap. 1.2, págs. impresas 7–19 → PDF 37–49.

Tema 8 — Tipos de sistemas operativos

Tanenbaum describe **nueve**, ordenados de la máquina más grande a la más chica.

Tipo	Qué lo caracteriza	Ejemplos

De mainframe	Lo que los distingue **no es la CPU, es la capacidad de E/S**: mil discos y millones de gigabytes. Tres servicios: **por lotes** (sin usuario presente), **procesamiento de transacciones** (muchos pedidos chicos, cientos o miles por segundo) y **tiempo compartido**.	OS/390
De servidores	Dan servicio a varios usuarios por red y permiten compartir recursos de hardware y software.	Solaris, FreeBSD, Linux, Windows Server
De multiprocesadores	Varias CPU en un sistema: computadoras en paralelo, multicomputadoras o multiprocesadores. Variantes de los de servidor con agregados de comunicación y consistencia.	Windows, Linux
De computadoras personales	Su trabajo es **dar buen soporte a un solo usuario**.	Linux, Windows, macOS
De bolsillo	Celulares y PDA, que se fusionaron. CPU de 32 bits con modo protegido.	Symbian, Palm OS
Integrados	Controlan dispositivos que no se consideran computadoras. Todo el software está en ROM.	QNX, VxWorks
De nodos sensores	Computadoras minúsculas a batería que se comunican por radio. SO chico y manejador de eventos.	TinyOS
De tiempo real	El **tiempo** es el parámetro clave. Ver abajo.	—
De tarjetas inteligentes	El más chico: un chip en una tarjeta. A veces una sola función, como el pago electrónico.	—

Tiempo real duro vs suave — esto se toma

```
TIEMPO REAL DURO                TIEMPO REAL SUAVE
----------------                -----------------
La acción DEBE ocurrir en       Es aceptable fallar
el instante exacto.             MUY ocasionalmente.
SIN EXCEPCIÓN.
Garantías ABSOLUTAS.            Sin garantías absolutas.
Si falla -> desastre físico     Si falla -> molestia

Ejemplos:                       Ejemplos:
. robot soldador en una línea   . audio digital
  de montaje                    . multimedia
. aeronáutica                   . teléfonos digitales
. control de procesos
. aplicaciones militares
```

El ejemplo del libro: *si un robot soldador suelda antes o después de tiempo, el auto se arruina.* En cambio, si la música se corta 20 milisegundos, molesta pero no pasa nada.

Integrados vs de bolsillo — la otra pregunta típica

```
DE BOLSILLO (celular)           INTEGRADO (microondas)
---------------------           ----------------------
Instalás apps de terceros       NO se instala nada.
                                Todo el software está en ROM.
         |                                |
         v                                v
Puede correr software           CERTEZA de que nunca correrá
NO CONFIABLE                    software no confiable
         |                                |
         v                                v
NECESITA protección             NO necesita protección
                                          |
                                          v
                                el diseño se SIMPLIFICA
```

Frase del libro: *la propiedad principal que diferencia a los sistemas integrados de los dispositivos de bolsillo es la certeza de que nunca se podrá ejecutar software que no sea confiable.*

Referencia: Tanenbaum, cap. 1.4, págs. impresas 33–37 → PDF 63–67.

Tema 9 — Estructuras de los sistemas operativos

Las cinco estructuras son **cinco respuestas a una sola pregunta: ¿quién corre en modo kernel?**
Más cosas en modo kernel → más rápido, pero un error tumba todo.
Menos cosas en modo kernel → más seguro, pero más lento.
Todo el tema es ese intercambio: rendimiento contra confiabilidad.

1. Monolítica — todo adentro

Es **un solo programa binario enorme**, todo enlazado, corriendo en modo kernel. Cualquier procedimiento puede llamar a cualquier otro directamente. Por dentro, tres niveles:

```
programa principal          <- recibe la llamada al sistema
       |
procedimientos de servicio  <- uno por cada llamada
       |
procedimientos utilitarios  <- ayudan a los anteriores
```

Ventaja: rapidísimo, llamar a otra parte del SO es una llamada a procedimiento común. **Desventaja:** un error en cualquier módulo **derriba el sistema entero**. Es la organización más común; Linux es monolítico.

2. Por capas — todo adentro, pero ordenado

```
+--------------------------+
|  5  el operador          |
|  4  programas de usuario |
|  3  administración de E/S|   <- acá viven los drivers
|  2  comunicación         |
|  1  memoria              |
|  0  procesador           |
+--------------------------+
      cada capa usa SOLO la de abajo
```

El primer ejemplo fue el **sistema THE**, con 6 capas. **MULTICS** lo generalizó con **anillos concéntricos**, donde los de adentro tienen más privilegios y llamar hacia adentro equivale a una llamada al sistema. **Ventaja:** claridad y depuración ordenada. **Desventaja:** no todo se deja ordenar en capas estrictas, y atravesarlas cuesta.

Ojo: las capas **no dan tolerancia a fallas**. Todo sigue corriendo en modo kernel, así que un driver que falla tumba el sistema igual que en la monolítica. Lo que mejora es la claridad para depurar.

3. Microkernel — casi todo afuera

```
+-----------------------------------------------+
|  MODO USUARIO                                 |
|  +--------+ +--------+ +--------+ +--------+ |
|  | driver | | driver | |servidor| |servidor| |
|  | video  | | disco  | |archivos| |  red   | |
|  +--------+ +--------+ +--------+ +--------+ |
+-----------------------------------------------+
+-----------------------------------------------+
|  MODO KERNEL                                  |
|        +------------------+                   |
|        |   MICROKERNEL    |  <- chiquito      |
|        +------------------+                   |
+-----------------------------------------------+
```

Solo el microkernel corre en modo kernel. Todo lo demás, **incluidos los drivers**, corre como procesos de usuario ordinarios sin privilegios. Un driver con errores puede corromper sus propios datos pero **no puede tumbar el sistema**: el SO lo detecta, lo mata y lo reinicia. Ejemplo: **MINIX 3**.

Importa porque los drivers son la causa más frecuente de caídas: es código de terceros con menos control de calidad que el kernel.

Regla de diseño: mecanismo en el kernel, política en el espacio de usuario. El kernel provee la capacidad de hacer algo; quién y cuándo se decide afuera.

Costo: lo que en un monolítico es un CALL, acá es **mandar un mensaje entre procesos**. Mucho más lento.

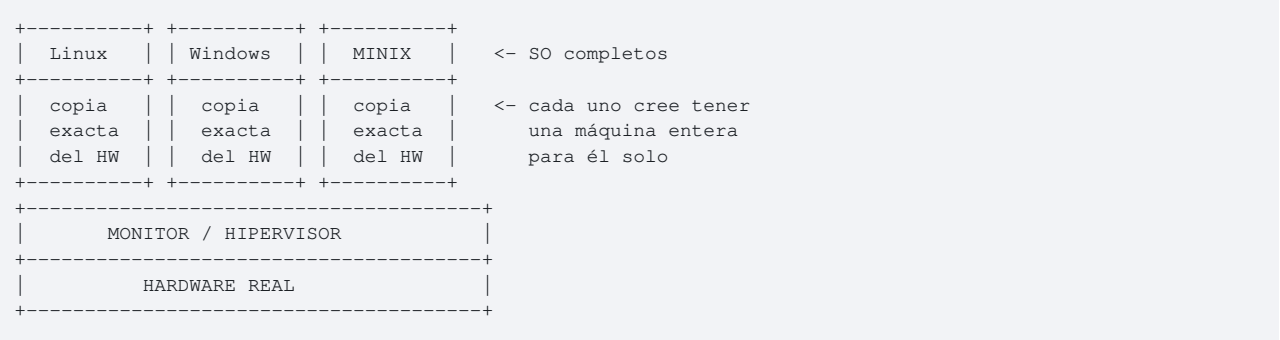
> **¿Cómo toca el hardware un driver que corre en modo usuario?** Haciendo una **llamada al sistema** al microkernel, que lo hace por él. Y el microkernel le da acceso **solo a los puertos de su dispositivo**, no a todo el hardware. Eso es exactamente "mecanismo en el kernel, política afuera".

4. Cliente-servidor — el microkernel, estirado

Variación leve del microkernel: **servidores** que proveen un servicio y **clientes** que lo usan, comunicados por **paso de mensajes**. Lo interesante: el cliente **no sabe** si el servidor está en la misma máquina o del otro lado de la red. Por eso es **la base conceptual de los sistemas distribuidos** (Unidad 8).

> **No confundir:** cliente-servidor como **estructura del SO** (cómo está organizado el kernel por dentro) es otra cosa que cliente-servidor como **arquitectura de aplicación** (tu app mandándole consultas SQL a un motor de base de datos). Son capas distintas del problema: un servidor de base de datos puede usar la arquitectura cliente-servidor a nivel aplicación y correr sobre un SO monolítico.

5. Máquinas virtuales — varios sistemas operativos a la vez



El monitor separa dos funciones que en los demás van juntas: la multiprogramación y la máquina extendida. **No ofrece una máquina cómoda**, ofrece **copias exactas del hardware real**, con su modo kernel, su modo usuario y sus interrupciones. El ejemplo clásico es el **VM/370** de IBM.

Hipervisor TIPO 1	Hipervisor TIPO 2
Corre sobre el **hardware** directamente. El monitor tradicional, VM/370.	Corre como un **programa de usuario** encima de un SO anfitrión, e interpreta el conjunto de instrucciones. VirtualBox sobre Windows.

La **JVM** es otra variante: una arquitectura de computadora definida por software, con un intérprete que ejecuta código byte. Por eso un programa compilado a código byte corre en cualquier máquina que tenga la JVM.

Exokernel: la idea opuesta a la máquina virtual. En vez de dar a cada usuario una copia de la máquina completa, le da **un subconjunto de los recursos reales** (por ejemplo, los cilindros 0 a 1023 del disco). Ahorra la capa de traducción entre direcciones virtuales y reales.

Comparación final

Estructura	¿Quién en modo kernel?	Ventaja	Desventaja	Ejemplo
Monolítica	Todo el SO	Rendimiento	Un error tumba todo	Linux, UNIX
Por capas	Todo, ordenado	Claridad	Difícil ordenar, costo por capa	THE, MULTICS
Microkernel	Solo el microkernel	**Confiabilidad**	Lento (mensajes)	MINIX 3, QNX
Cliente-servidor	Microkernel + mensajes	Se extiende a la red	Latencia	Base de los distribuidos
Máquinas virtuales	El hipervisor	Aislamiento total	Sobrecarga	VM/370, hipervisores

Linux hace trampa, y es la respuesta inteligente: es monolítico, pero con **módulos cargables** —pedazos de código, típicamente drivers, que se insertan y se quitan del kernel **en caliente**, sin reiniciar—. Un término medio entre el monolito rígido y el microkernel.

La regla para elegir: ¿qué me duele más perder? Rendimiento → **monolítica**. Confiabilidad → **microkernel**. En tiempo real duro la respuesta es casi siempre microkernel, por la misma razón por la que es duro: no puede fallar. (QNX, un microkernel, es el SO de aviónica, sistemas médicos y autos.)

Referencia: Tanenbaum, cap. 1.7, págs. impresas 62–72 → PDF 92–102.

Cierre — los 8 puntos que hay que tener firmes

1. **El SO hace dos cosas que no tienen que ver entre sí.** Máquina extendida (abstracciones, la principal es el archivo) y administrador de recursos (*asignación ordenada y controlada*). Multiplexa en el **tiempo** (CPU) y en el **espacio** (memoria, disco).
2. **Modo kernel y modo usuario es el cimiento de todo.** Un bit del PSW decide. Las instrucciones privilegiadas **las rechaza el hardware antes de ejecutarlas**, no el SO después.
3. **Los tres métodos de E/S tardan lo mismo. Lo que cambia es la CPU:** 100 % / 5 % / 0,005 %. La trampa es preguntar cuál es "más rápido": ninguno.
4. **La cadena de arranque es una cadena de cargadores**, cada uno más grande que el anterior y existiendo solo para cargar al siguiente: BIOS → MBR (446+64+2) → partición activa → cargador secundario → SO.
5. **INT elige la familia, AH elige la función, los otros registros llevan los datos.** INT 21h / AH=09h muestra una cadena desde DS:DX terminada en `$`; AH=02h muestra un carácter desde DL.
6. **La llamada al sistema es la única puerta al kernel.** TRAP se diferencia de un CALL en dos cosas: cambia el modo y salta a una dirección fija. Esas dos diferencias son toda la seguridad.
7. **La historia es una cadena de agujeros de tiempo muerto que se van tapando.** Y la distinción clave: **lotes = uno en memoria por vez, en fila; multiprogramación = varios en memoria a la vez**. La multiprogramación exige hardware de protección.
8. **Las cinco estructuras responden a: ¿quién corre en modo kernel?** Y la respuesta es siempre un intercambio: monolítico = rápido y frágil; microkernel = lento y robusto.

Lo que más se toma de esta unidad

Casi seguro que cae

strace — la sintaxis exacta: `strace -o salida.txt ls -la`. Distractores habituales: `>`, `|`, `-s`.

Comandos Linux de las cartillas:

Comando	Para qué
`ls -l /dev | grep '^c'`	identificar dispositivos de **carácter** (con `^b`, de bloque)
`pgrep firefox` / `ps -ef | grep firefox`	obtener el **PID** de un proceso
`vmstat`	columna **id** = % de tiempo que la CPU está **inactiva**
`free -h` / `cat /proc/meminfo`	RAM total y libre
`lsblk -f`	dispositivos de bloque, espacio usado y libre
`cat /etc/fstab`	**tamaño de bloque** de una partición ext4
`kill -STOP PID` / `kill -CONT PID`	suspender y reanudar un proceso
`iostat`	actividad, velocidad, espera y utilización de E/S

INT 21h — completar el código para mostrar una cadena: AH=09h, la cadena con `$`, DS:DX.

Aparece seguido

- Las tres formas de E/S y cuál conviene según el dispositivo
- Modo kernel vs usuario y qué pasa con una instrucción privilegiada
- Lotes vs multiprogramación (verdadero/falso con justificación)
- Tiempo real duro vs suave, con ejemplos

Más teórico, suele ser desarrollo

- Las cinco estructuras y su intercambio rendimiento/confiabilidad
- Historia y por qué aparece cada invento

Los cuatro errores a no repetir

1. **"El SO deja de responder"** ante una instrucción privilegiada → **el hardware la rechaza** antes de ejecutarla.
2. **Los 64 bytes "son" las particiones** → la tabla **describe**, no contiene. 16 bytes por renglón, sea de 50 MB o de 2 TB. Tabla completa de 4 particiones = 64 bytes.
3. **Cliente-servidor para algo sensible a la latencia** → es **el más lento** de todos. Y no confundir la estructura del SO con la arquitectura de una aplicación.
4. **Capas para tolerancia a fallos** → en capas **todo sigue en modo kernel**: un driver que falla tumba todo igual.

Las cuatro visualizaciones interactivas

Todas accesibles desde `visuales/index.html`:

- **es_programada_interrupciones_dma.html** — las tres formas de E/S, con línea de tiempo y costo de CPU calculado.
- **arranque_y_sector_de_booteo.html** — el arranque paso a paso; se puede cambiar la partición activa.
- **interrupciones_8086.html** — simulador de INT 10h/16h/17h/21h con pantalla de DOS y detección de los errores típicos.
- **estructuras_de_so.html** — las cinco estructuras y qué pasa en cada una cuando falla un driver.